



API Gateway Pattern Essentials

Understanding Gateways in Modern Systems



What Is an API Gateway?

Unified Access Point

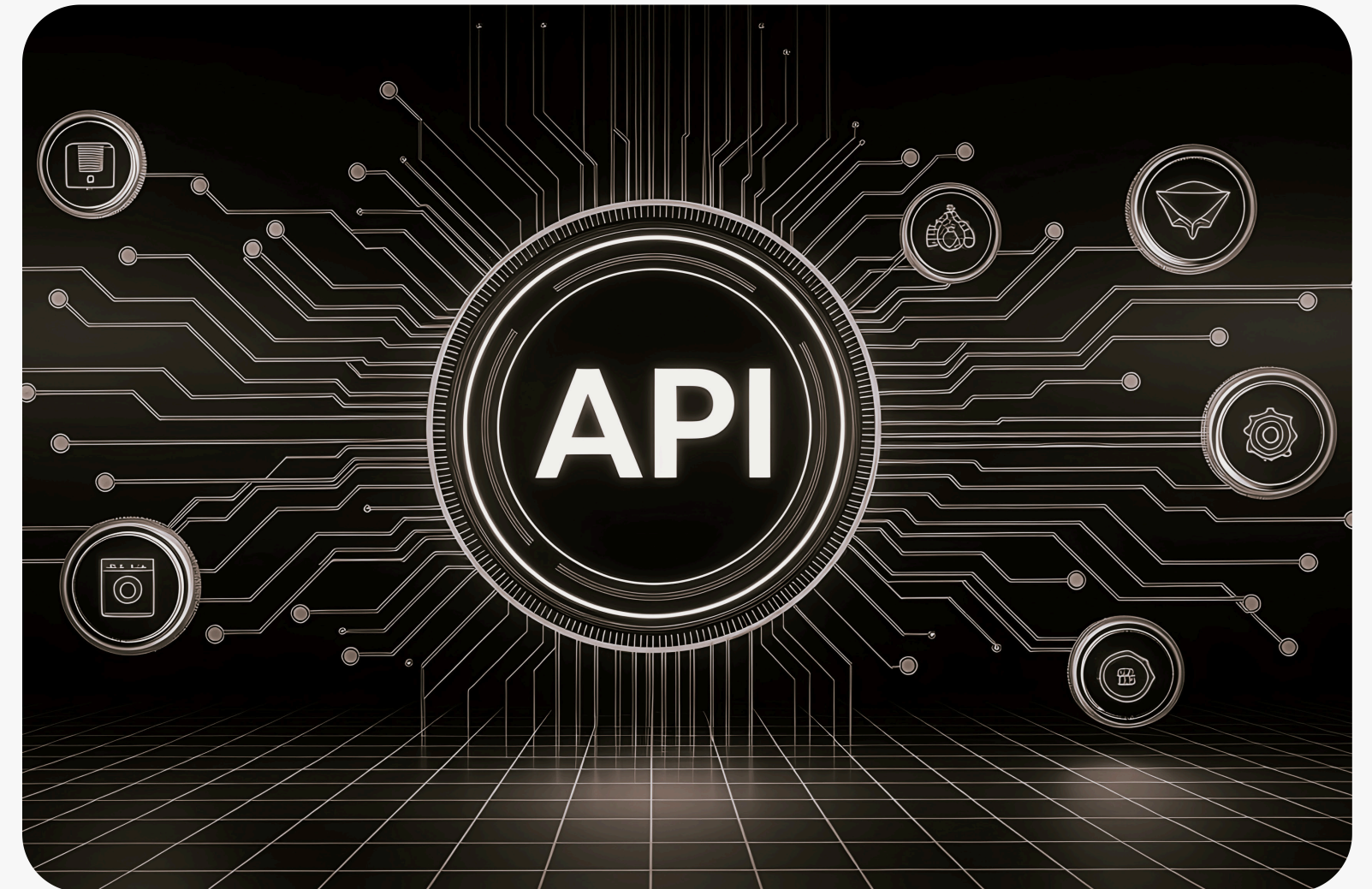
Clients interact with one gateway instead of many services.

Efficient Request Routing

Gateway directs client requests to the correct backend service.

Simplified Client Experience

Reduces complexity for clients accessing microservices.





Why API Gateway Is Needed



Conceals Service Complexity

Clients interact with a unified interface, not individual services.



Centralized Security Controls

Authentication and authorization managed in one place.



Efficient Request Routing

Directs client requests to the correct backend service.



API Gateway Responsibilities

Centralized Authentication

Validates incoming requests before reaching services.

Rate Limiting Enforcement

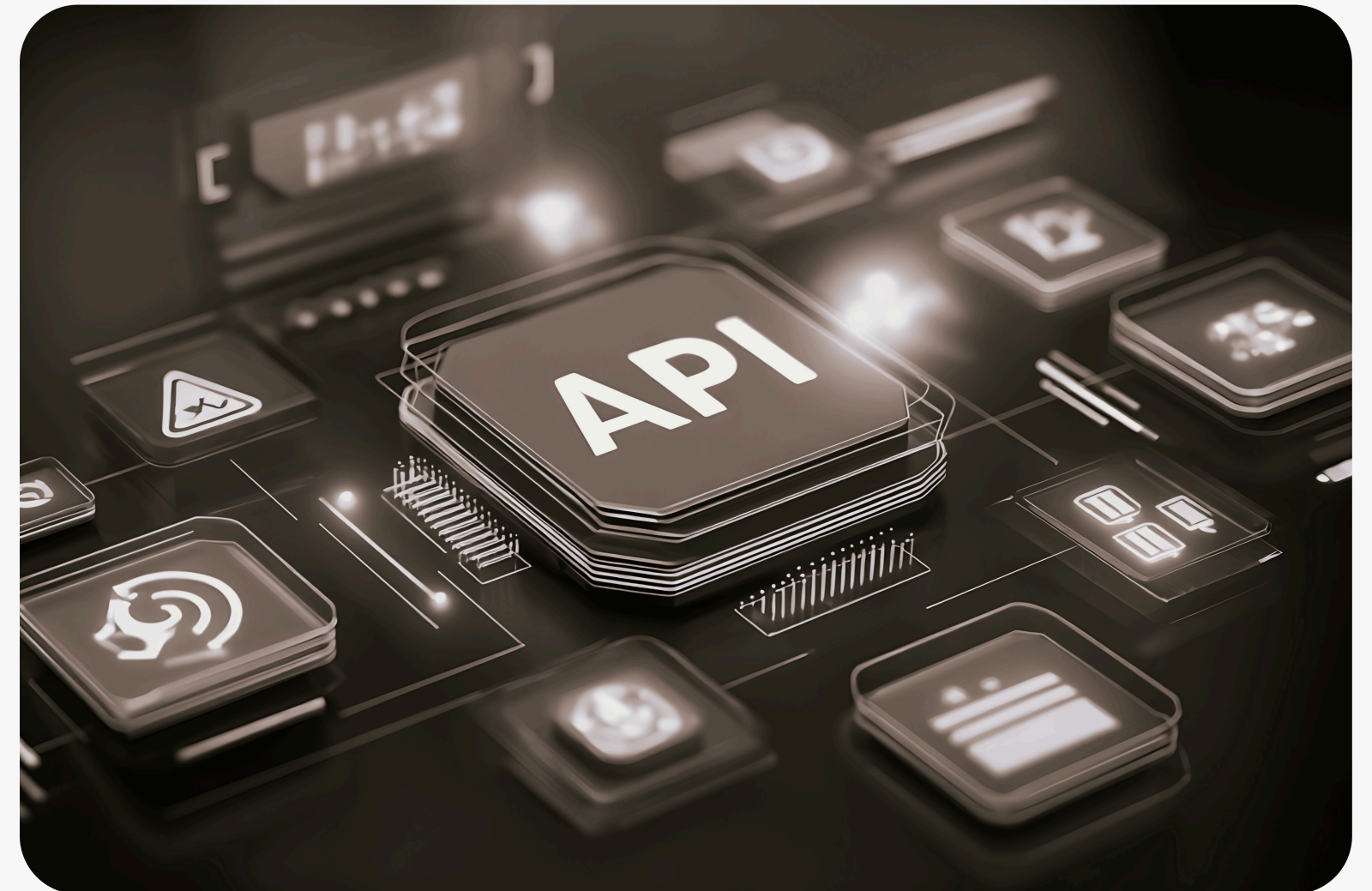
Controls traffic to prevent service overload.

Request Routing & Aggregation

Directs and combines requests for client simplicity.

Separation from Business Logic

Keeps core business rules out of the gateway.





Building a Custom Gateway



Custom Gateway Development

Leverage Node.js or NestJS for implementation.



Reverse Proxy Functionality

Directs client requests to backend services.



Middleware for Flexibility

Integrate authentication, logging, and more.



Backend for Frontend (BFF)



Dedicated backend for each client

Ensures APIs are optimized for specific client needs



Tailored APIs for mobile, web, admin

Delivers relevant data and functionality per interface



Solves over-fetching and under-fetching

Prevents clients from receiving too much or too little data



API Gateway Best Practices



Design for Simplicity

Keep the gateway thin and stateless for scalability.



Separate Business Logic

Avoid embedding business rules in the gateway.



Prevent Gateway Bloat

Ensure the gateway doesn't become a mini-monolith.